

Mini Pokémon-Style Battle Game Project Plan

This document outlines the planned systems, development tasks, architecture ideas, and important considerations for a 2D pixel-art Pokémon-style battle simulator built with Pygame. The game is inspired by Pokémon Emerald battles but focused only on local battle gameplay, not overworld RPG exploration.

Core Vision

- 2D pixel-art battle simulator inspired by Gen 3 Pokémon Emerald.
- Local-only gameplay (no online multiplayer).
- Around 30 Pokémon initially.
- Maximum of ~50 moves.
- Custom move animations and pixel-art effects.
- Team builder with Pokémon selection, moves, items, abilities, natures, and evolutions.

Main Systems To Build

- Main menu system.
- Team builder interface.
- Battle scene renderer.
- Turn-based combat engine.
- Move selection menus.
- Damage calculation system.
- Type effectiveness system.
- Status effects system.
- AI opponent logic.
- Animation queue system.
- Sound and music handling.
- Save/load system for teams.

Recommended Development Order

- Phase 1: Create a basic battle prototype with 2 Pokémon and 2 moves.
- Phase 2: Add move selection, HP bars, and switching.
- Phase 3: Add team builder functionality.
- Phase 4: Add abilities, items, and natures.
- Phase 5: Add polished animations, sounds, and visual effects.
- Phase 6: Expand roster to 30 Pokémon and ~50 moves.

Battle Engine Considerations

- Use a turn queue system instead of hardcoded move interactions.
- Separate game logic from rendering and animation.

- Build reusable move effect systems.
- Design move resolution in stages: selection, priority, damage, effects, faint checks.
- Avoid writing unique code for every move.

Data Architecture

- Use JSON files for Pokémon, moves, abilities, and items.
- Avoid hardcoding move names or Pokémon-specific behavior.
- Keep stats, move power, effects, and typing data external from battle code.
- Use modular classes for Pokémon, moves, items, and battle actions.

Pixel Art & Visual Design

- Use a low internal resolution such as 320x180 for crisp pixel scaling.
- Use 64x64 or 96x96 Pokémon sprites.
- Reuse particle systems and animation components between moves.
- Use spritesheets instead of many separate image files.
- Focus on readability and clarity during battles.

Move Animation Ideas

- Projectile animations for ranged attacks.
- Screen shake and sprite lunges for physical attacks.
- Particle systems for status and buff moves.
- Beam rendering for laser-style attacks.
- Impact flashes and hit sparks.

Audio Considerations

- Battle music.
- Menu sounds.
- Attack sound effects.
- Damage impact sounds.
- Fainting sounds.
- Victory music.

Performance & Technical Concerns

- Cache images and sounds.
- Avoid loading assets every frame.
- Use delta time or frame-independent animation timing.
- Keep animation logic modular.
- Organize project folders cleanly from the start.

Scope Control

- Do not attempt to recreate every Pokémon mechanic.
- Limit the number of Pokémon and moves initially.
- Finish a polished core battle experience before expanding.
- Prioritize gameplay completion over visual perfection.
- Avoid feature creep.

Suggested Folder Structure

- main.py
- battle.py
- renderer.py
- ui.py
- pokemon/
- assets/sprites/
- assets/effects/
- assets/audio/
- data/pokemon.json
- data/moves.json

Recommended Tools

- Pygame for game engine framework.
- Aseprite or Piskel for pixel art.
- JSON for game data storage.
- Git for version control.

The most important goal is to complete one fully playable battle loop first. A clean architecture and controlled scope will matter more than adding huge amounts of content early.